Для всех заданий:
1) собирать при помощи make или cmake;
2) прикреплять ссылку на github с исходным кодом программы;
3) все вычисления проводить на сервере.
4) отчет прикреплять в google class

Анализ эффективности

- Введем обозначения:
  - $\boldsymbol{T(n)}$ – время выполнения последовательной программы (serial program) при заданном размере *n* входных данных
  - $\boldsymbol{T_p(n,p)}$ – время выполнения параллельной программы (parallel program) на *p* процессорах при заданном размере *n* входных данных

- Коэффициент $S_p(n)$ ускорения параллельной программ (Speedup):

$$\boldsymbol{S_p(n) = \frac{T(n)}{T_p(n)}}$$

Во сколько раз параллельная программа выполняется на *p* процессорах быстрее последовательной программы при обработке одних и тех же данных размера *n*

- Как правило

$$S_p(n) \leq p$$

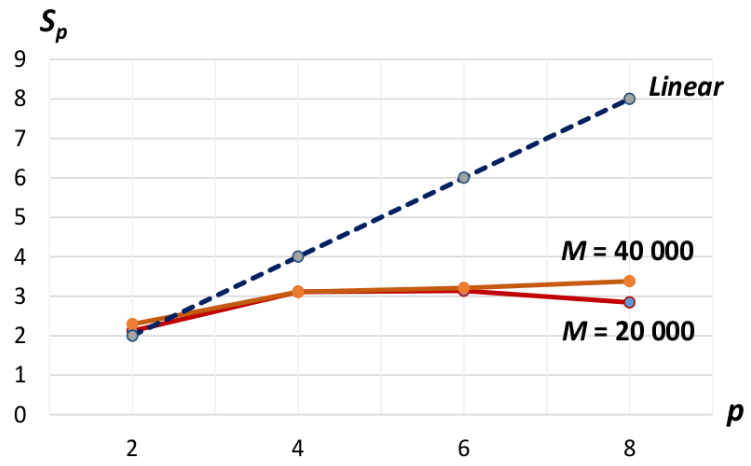
- Цель распараллеливания –
  достичь линейного ускорения на наибольшем числе процессоров: $S_p(n) \geq c \cdot p,$ при $p \to \infty$ и $c > 0$

Задание 1

Используя std::thread, std::jthread или асинхронные задания, реализовать многопоточную версию программы умножения матрицы на вектор с параллельной инициализацией массивов (Код на C++). Провести анализ маштабируемости, (заполнить таблицу, по шаблону ниже):

M = N	Количество потоков								
	2			4		6		8	
	T_1	T_2	S_2	T_4	S_4	T_6	S_6	T_8	S_8
20 000 (~ 3 GiB)									
40 000 (~ 12 GiB)									

Заполнить таблицу для 1,2,4,7,8,16,20,40 потоков. Для элементов массива используйте тип double. Размеры матриц: 20000x20000, 40000x40000. Построить график ускорения в зависимости от количества потоков:



Написать вывод о маштабируемости. Результат сохранить в формате pdf, и прикрепить в гугл класс. *Дополнительно* (по желанию): Привязать потоки к ноде или конкретным ядрам, и сравнить результат с предыдущим (без привязки).

Задание 2

Реализовать клиент-серверное приложение.

Сервер представляет из себя отдельный поток, который берет задачи из очереди, решает их и сохраняет результат в контейнер (подберите оптимальный контейнер, в который можно быстро добавлять и удалять элементы, а также искать их). Реализовать в виде шаблона класса. У серевера обязательно должны быть следующие функции-члены (интерфейс):

1) void start() - запустить сервер;

2) void stop() - остановить сервер;

3) size_t add_task(task) - добавить задачу, тип task в параметрах функции на ваше усмотрения. Возвращает id задачи;

4) T request_result(id_res) - возвращает результат типа T. Может быть блокирующим или не блокирующим.

Клиенты представляют из себя отдельные потоки, которые добаляют задачи в очередь, дожидаются результата и сохраняют в файл. Создать три клиента, каждый добавляет по N (5<N<10000) задач. Для каждого клиента использовать следующие задачи (каждый клиент добавляет только один тип задач):

1) вычисление синуса, аргумент задавать рандомно;

```
1  template<typename T>
2  T fun_sin(T arg)
3  {
4      return std::sin(arg);
5  }
```

2) вычисление квадратного корня, аргумент задавать рандомно;

```
1  template<typename T>
2  T fun_sin(T arg)
3  {
4      return std::sqrt(arg);
5  }
```

3) возведение числа в степень, аргументы задавать рандомно.

```
template<typename T>
T fun_sin(T x, T y)
{
    return std::pow(x, y);
}
```

Результаты в файлах должны быть удобны для чтения и проверки. Написать тест, который читает результаты предыдущей программы из файлов и проверяет.

Warning: При работе с общими ресурсами (очередь, контейнер, переменные) из разных потоков, не забывайте синхронизировать доступ к ним. Метод get() у future можно вызывать только один раз.

Примеры передачи задач в отдельный поток: example_add_async.cpp, example_add_package.cpp.

Дополнительно (по желанию): Реализовать Thread Poll (плюс 3 балла).